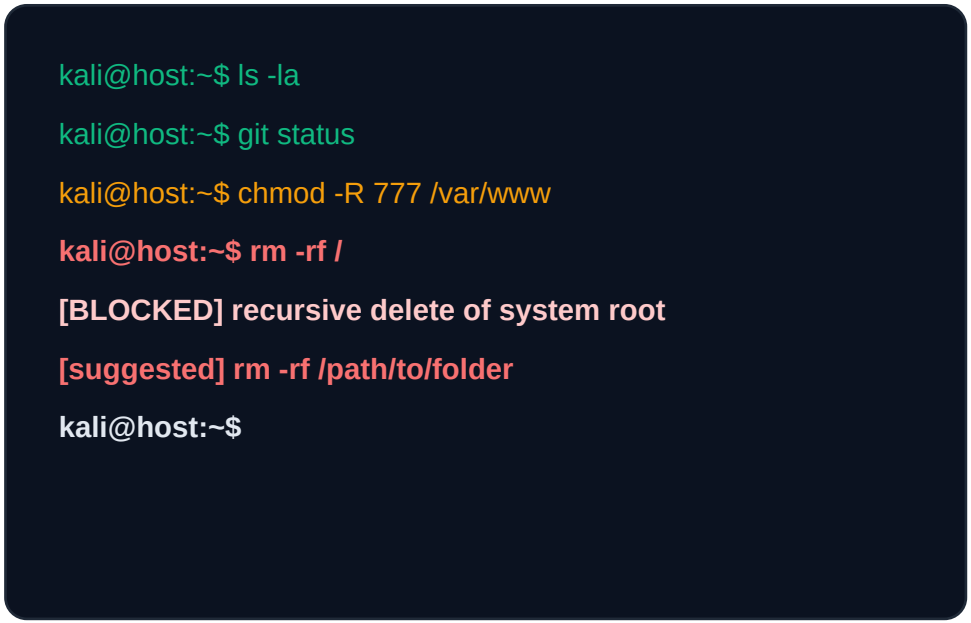
SAFESHELL · COMMAND SAFETY ENGINE

Transactional execution with AI-generated undo plans

C-DAC Secure OS Hackathon 2026 · Track 1: AI at Application Level



Adhithya J

github.com/Adhi-hub07/command-safety-engine

Demo: csengine run "rm -rf projects" → csengine undo last

1. The problem

every terminal keystroke can be the last

- A single destructive command is often irreversible — and typed by hand or pasted from the web.
- Fork bombs, “download-and-run” chains, chmod 777, credential scraping, obfuscated payloads.
- No mainstream OS ships a safety layer between the keyboard and the kernel.

Why existing tools fall short

sudo / SELinux
stop unauthorised actions, not user mistakes

blocklists
brittle strings, false-positive fatigue

cloud AI helpers
can't run on sovereign / air-gapped systems

Challenge (verbatim)

SafeShell: design a secure command execution framework that generates AI-based rollback plans, simulates command execution, and provides recovery mechanisms to safely execute system operations.

Our answer

Detect intent before execution — then make it reversible: snapshot every target, simulate the command, and undo any damage with one command.

2. Solution overview

four-layer defence-in-depth pipeline

- Defence in depth: four independent layers, any layer can only push toward more caution.

On the happy path: zero overhead

whitelisted daily commands skip analysis entirely

Layer 2 · ML Classifier

GradientBoosting on 20 intent features, 3 classes (safe / risky / destructive).

Layer 3 · LLM Explainer

Qwen 2.5 3B via Ollama, 100% offline, only for ambiguous commands.

Layer 4 · Transaction & Recovery

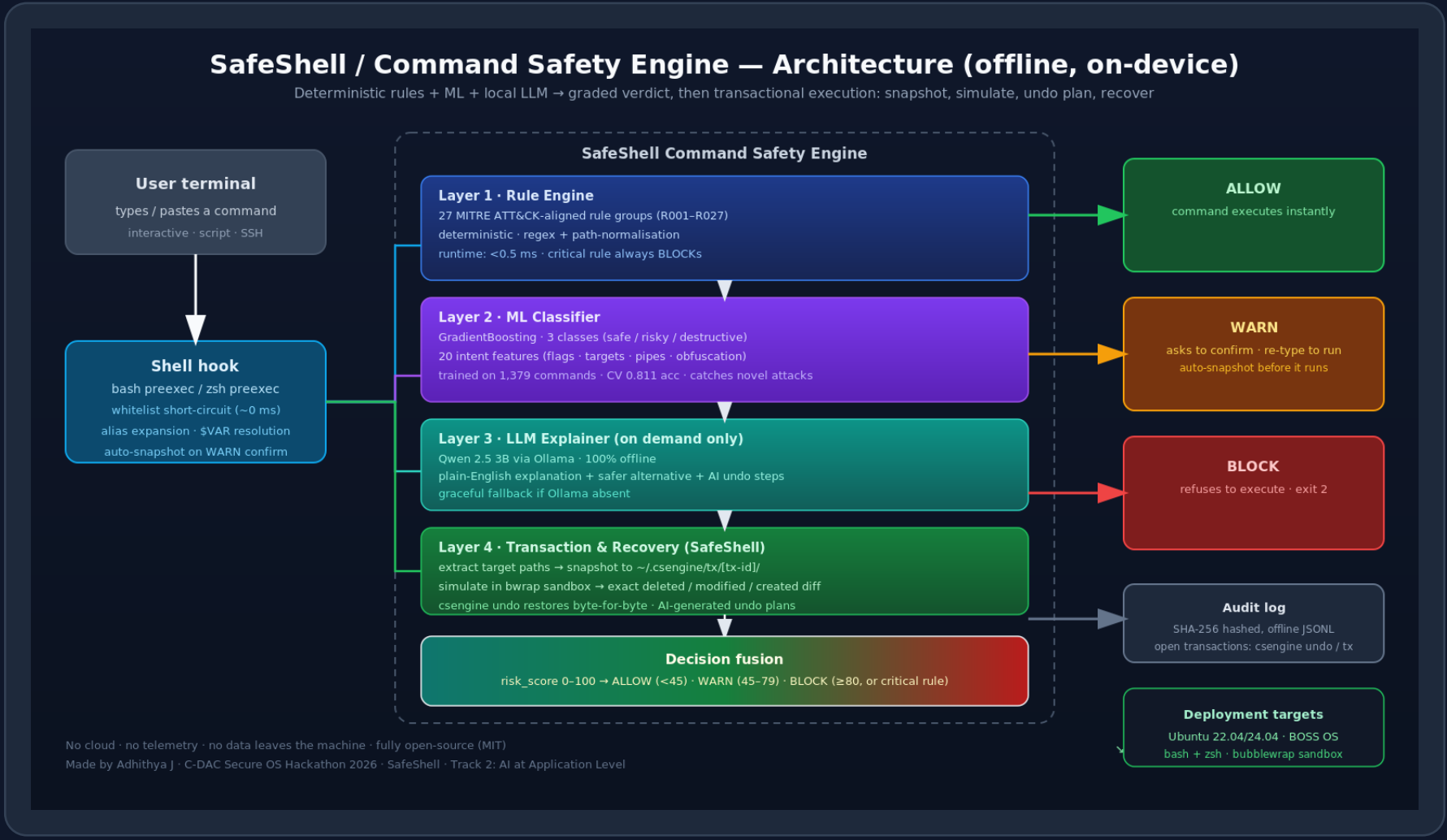
snapshot targets · simulate in bwrap sandbox · AI undo plans · csengine undo restores.

Fusion & output

`rule_score`	max severity of matched rules (0–100)
`ml_risk`	label risk × confidence
`score`	blend → 0–100
`BLOCK`	≥ 80 or critical rule
`WARN`	45–79, asks to confirm
`ALLOW`	< 45 or whitelisted

3. Architecture

offline, on-device, transactional



4. Layer 1 — deterministic rules

27 MITRE ATT&CK-aligned rule groups

- Fast, deterministic first gate — everything else builds on it. Anchored, escaped regexes, not naive substrings.

Destructive / data loss

R001 rm -rf root · R023 system-file tamper · R024 block-device write · R025 tar --remove-files · R027 --no-preserve-root

Denial of service

R008 fork bomb · R026 git clean · R021 variable delete

Credential exposure

R022 /etc/shadow · /etc/sudoers · ssh keys

Supply chain / download-and-run

R004 curl|bash · wget|sh · curl -o + execute

Escalation / integrity

R007 chmod -R 777 · R017 kill/shutdown · R030 sy/sudo misuse

Safety-first override

critical rule ⇒ BLOCK regardless of ML confidence

5. Layer 2 — machine learning

generalises to attacks no rule can enumerate

- GradientBoosting on 20 hand-crafted intent features — generalises to attacks no rule can

Data hygiene

666 duplicate rows removed at build; numbers are dedup, not cherry-picked

ork calls | disk ops | wildcards

recursive flags | chmod 777 | sudo | env-var manipulation

Measured results (honest, reproducible)

5-fold CV 0.811 ± 0.033 acc · 0.771 ± 0.048 macro-F1 · held-out 82.2% / 0.791

Model	Acc	Macro-F1	Destr. recall	Risky recall	Safe recall
`GradientBoosting (deployed)`	0.822	0.791	0.771	0.632	0.934
`RandomForest`	0.794	0.772	0.771	0.724	0.836
`LogisticRegression`	0.815	0.777	0.688	0.645	0.941

6. Layer 3 — LLM explainer

plain-language answers, still fully offline

- Layer 3 exists for one reason: explain the decision in plain language when the command is confusing.
- Local Qwen 2.5 3B via Ollama — no cloud, no network, no data egress.
- Invoked only on demand (ambiguous / flagged / LLM hook), so idle cost is
- 0.5 s TCP probe: on absence the rule+ML explanation is returned instantly

Example — obfuscated command

```
$ xxd -r -p <<< '77686f616d69'
```

Decoding a hex blob is how malware hides. The LLM explains: “this decodes the text ‘whoami’ — a read-only command.”

Privacy & audit

- commands never stored in plaintext
- SHA-256 hashed JSONL audit log
- offline by design, no telemetry
- override is logged and detectable

forensics without storing plaintext

7. SafeShell — transactional execution & recovery

snapshot · simulate · undo plan · recover

- Blocking is not enough — confirmed risky commands must be reversible. SafeShell wraps them in a transaction.

1 · Path extraction	parses rm / mv / cp / chmod / chown / dd / touch / tee + redirect targets from the AST
2 · Undo plan	deterministic offline steps (restore / move-back / chmod-back) merged with LLM recovery steps
3 · Snapshot	every target saved with content, mode, owner, symlink under ~/.csengine/tx/[tx-id]/
4 · Simulation	bwrap sandbox dry-run on directory copies → exact deleted / modified / created diff
5 · Recover	csengine undo restores everything byte-for-byte — a fatal rm -rf becomes an inconvenience

Demo: csengine run "rm -rf projects" → undo plan + simulated diff → csengine undo last → file back

auto-snapshot in bash/zsh hooks when a WARN command is re-typed to confirm (CSENGINE_TX=0 disables)

8. Shell integration & user control

the hook, the verdicts, the exits

- Installs in minutes on Ubuntu / BOSS OS via a single script (setup.sh).
- Hooks into bash and zsh (preexec) — every command is judged before it r
- Rich CLI: csengine check <cmd> · csengine run · csengine undo · csengin
- WARN commands auto-snapshot on confirmation; CSENGINE_TX=0 disal

ALLOW

daily commands execute instantly

whitelist or score < 45

WARN

asks to confirm / retype + snapshot

score 45–79

BLOCK

refuses execution, exit 2

score ≥ 80 or critical rule

Exit codes & scripting

```
exit 0 → ALLOW (executed)
exit 1 → WARN (confirmed to run)
exit 2 → BLOCK (refused)
JSON output for automation
```

9. Performance & feasibility

measured, not promised

- Measured on a Kali VM (scripts/benchmark_latency.py --n 1000):

whitelisted happy path	`~0 ms`
feature extraction + rule check	`~0.3 ms mean · ~0.5 ms p95`
full pipeline (no LLM resident)	`~2.7 ms mean · ~3 ms p95`
full pipeline (LLM pre-warmed)	`tens of ms`
simulation of a risky command	`~0.07 s (small target trees)`

Model training

trains on CPU in ~1 s · runs in ~0.1 ms · no GPU · CI retrains on every push

Why GradientBoosting, not deep learning?

Same accuracy at this data scale, trains on a laptop, fully explainable — judges and auditors can read exactly which features fired.

Feasibility for target OS

- runs on 4 GB RAM / 2 vCPU
- BOSS OS (sovereign GNU/Linux) ready
- bubblewrap sandbox optional
- open-source: Qwen 2.5, scikit-learn, Ollama

10. Risks & mitigations

honest assessment, engineered in

Risk	Severity	Mitigation
Adversarial bypass via encoding	High	odes base64/hex · obfuscation features feed ML · audit-logged
		ional layer · rule+ML always answers in <60 ms · status surfaces it
False positives	Med	whitelist short-circuit · anchored rules · graded WARN not hard block
Model drift	Low	CI retrains + 5-fold CV printed on every push
User disables protection	Low	overrides hashed in audit; repeat BLOCK-overrides detectable

Fail-safe, not fail-open

any layer can only push the decision toward more caution

11. Innovation & mission alignment

why this is different

- **Transactional command execution**
  risky commands become reversible — snapshotted before run, restored byte-for-byte by csengine undo.
- **Simulation-based safety guarantees**
  the exact deleted / modified / created diff is computed in a bwrap sandbox and shown before anything runs.
- **AI-generated undo plans**
  deterministic offline recovery steps merged with LLM recovery steps; recovery never depends on a model.
- **Safety-first critical override**
  one critical rule match blocks regardless of ML confidence.
- **Fully offline, sovereign-first**
  no cloud, no egress; deployable on BOSS OS, aligned with India's AtmaNirbhar mission.

C-DAC mission alignment

open-source · sovereign OS · no foreign cloud AI · improves govt/enterprise security posture

12. Roadmap & status

from prototype to sovereign default

- Beyond the hackathon:
  - **BOSS OS packaging** — .deb / .rpm for major distributions
  - **Hindi-first UI** — explanations in more languages
  - **Personal allowlists** — per-user adaptive trust
  - **Transaction lifecycle** — auto-cleanup of stale snapshots

Status

code complete · 66/66 tests · 17/17 demo checks · bash+zsh pty hooks pass

Thank you

Live demo: csengine check “rm -rf /” → BLOCK

github.com/Adhi-hub07/command-safety-engine